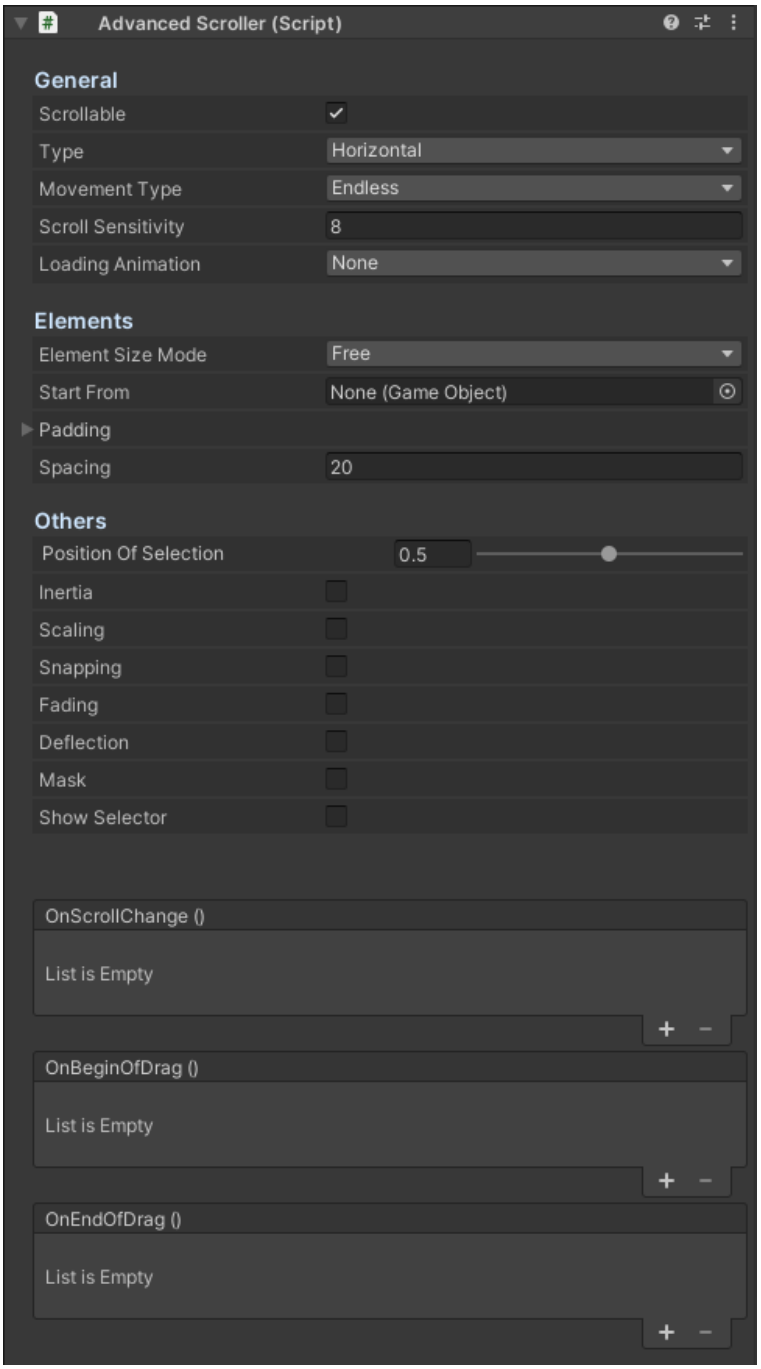
Advanced Scroller for Unity

Welcome to Advanced Scroller, the ultimate scrolling solution for your Unity projects! Advanced Scroller takes the concept of scrolling to a whole new level, offering you enhanced control, versatility, and customization options to create captivating and interactive scrolling experiences. Say goodbye to basic scrolling and embrace the power of Advanced Scroller to elevate your Unity projects. Scroll smarter, scroll better, and scroll like a pro with Advanced Scroller.

By using Advanced Scroller, you agree to and have read the End User License Agreement (EULA), which governs the terms and conditions of its usage.



1. How to create Advanced Scroller **3**
2. General **3**
Scrollable 3
Type 3
Movement Type 3
Scrolling Sensitivity 4
Loading Animation 4
● Available Animations: 4
General settings for Loading animations 4
Specific settings for Loading animations 4
3. Elements **5**
Element Size Mode 5
Alignment / Start From 5
Padding 6
Spacing 6
4. Others **7**
Position of Selection 7
Inertia 7
Scaling 8
Snapping 9
Fading 9
Deflection 10
Mask 11
Show Selector 12
5. Useful methods **13**
horizontalNormalizedPosition 13
verticalNormalizedPosition 13
ScrollToElement 13
SelectedElement 13
ScrollToNextElement 14
ScrollToPreviousElement 14
DoScroll 14
LoadScroller 14
Feedback and Support **15**
Found a Bug or Need Help? **15**

1. How to create Advanced Scroller

Right click in hierarchy > UI > Advanced Scroller or by dragging prefabs of AdvancedScroller to the scene located in Assets > AdvancedScroller > Prefabs.
Elements should be added to the ElementsContainer. These elements will be scrollable within the scroller.

2. General

To access the Advanced Scroller, you need to use the AScroller library.

```
using AScroller;
```

Scrollable

The "Scrollable" boolean in Advanced Scroller allows you to enable or disable input for scrolling,

```
AdvancedScrollerRef.Scrollable = false;
```

Type

The "Type" enum in Advanced Scroller offers two options: "Horizontal" and "Vertical." This setting allows you to specify the scrolling direction for your scroller, ensuring it behaves exactly as intended in your Unity project.

```
AdvancedScrollerRef.Type = type.Horizontal;
```

Movement Type

The "Movement Type" enum in Advanced Scroller offers three distinct options: "Endless," "Clamped," and "Elastic." This setting determines how your scrolling elements behave within the scroller, allowing you to choose the ideal movement style.

- **Endless:** enables endless scrolling, where elements repeat seamlessly as you scroll in one direction.(elements are not instantiated repeatedly)
- **Clamped:** Choose "Clamped" to restrict scrolling within a specified width or height. This setting ensures that scrolling is limited to a defined boundary.
- **Elastic:** Opt for "Elastic" to allow scrolling beyond the scroller's boundaries. When scrolling stops, it gently snaps back to the nearest edge. When you select "Elastic" as the movement type, the **Elasticity** float value becomes available for adjustment. This value determines the speed at which the content snaps back to the nearest edge after overscrolling.

```
AdvancedScrollerRef.MovementType = movementType.Endless;
```

Scrolling Sensitivity

The "Scrolling Sensitivity" setting allows you to fine-tune the scroll speed of your scroller. This numeric value influences how quickly your content scrolls in response to user input.

```
AdvancedScrollerRef.ScrollSensitivity = 10f;
```

Loading Animation

The "Loading Animation" setting in Advanced Scroller introduces dynamic loading animations to enhance the visual appeal of your Unity project. This versatile enum offers a variety of loading animations, each with its unique characteristics and customizability.

- **Available Animations:**

  Fade, Continuous Fade, Scale, Move.

  Fade: smoothly fades in the entire scroll content,
  Continuous Fade: elements fade in continuously, with each subsequent element starting to fade only after the previous one has completed its fade-in transition,
  Scale: gradually scales elements from a specified scale factor to 1,
  Move: gracefully shifts elements from a specified position to 0.

General settings for Loading animations

Play On Start: When enabled, the loading animation automatically starts when the scene begins, smoothly transitioning into the content,
Delay: Set the delay duration (in seconds) to specify when the loading animation commences after scene start,
Ease: Choose from a range of easing functions, such as linear, outback, inback, and more, to define the curve along which the fade-in/scale/move occurs,
Duration: Determine the duration (in seconds) of the fading/scale/move animation, allowing you to control the speed..

Specific settings for Loading animations

Scale Animation: you can define a starting scale (Vector2) from which the elements smoothly transition to a scale of 1,
Move Animation: allows you to specify a starting position (Vector2) from which the elements move to the final position at 0.

Loading Animation Examples:

```
AdvancedScrollerRef.LoadingAnimation = loadAnim.fade;
AdvancedScrollerRef.LoadingAnimPlayOnStart = true;
AdvancedScrollerRef.LoadingAnimDelay = 0.5f;
AdvancedScrollerRef.LoadingAnimDuration = 1f;
AdvancedScrollerRef.LoadingAnimEase = ease.Linear;
AdvancedScrollerRef.LoadingAnimMove = Vector2.one;
AdvancedScrollerRef.LoadingAnimScale = Vector2.zero;
```

To initiate the loading animation for the Advanced Scroller dynamically from your script, you can use the LoadScroller method along with a specified delay:

```
AdvancedScrollerRef.LoadScroller(delay);
```

3. Elements

Element Size Mode

allowing you to control the size elements. This setting can have two values: "Free" and "Fixed."

```
AdvancedScrollerRef.ElementsSizeMode = sizeMode.Fixed;
```

Free Mode: In this mode, you have the flexibility to adjust the width and height of the elements by yourself in the RectTransform component of each element. By hovering the mouse over the Advanced Scroller script in the Unity Inspector, the script should automatically update elements position according to sizes and spacing.
Fixed Mode: By selecting Fixed mode, a Vector2 field **Element Size** becomes available. Here, you can precisely define the width and height of the elements. Once these values are set, all elements within the Advanced Scroller will conform to the dimensions provided in the Element Size field. Additionally, the Advanced Scroller will automatically adjust the elements' positions according to the specified sizes and spacing.

```
AdvancedScrollerRef.ElementsSize = new Vector2(100, 100);
```

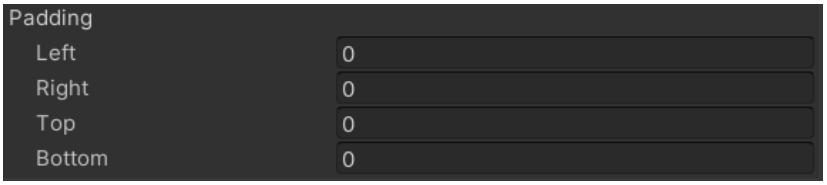
Alignment / Start From

The Alignment / Start From settings in the Advanced Scroller are closely related to the chosen Movement Type. Depending on your selection for Movement Type, these settings will offer different options:

Alignment: If Movement Type is set to Elastic or Clamped, the Alignment setting becomes active, while Start From is disabled. The Alignment is an enum with values such as Center, Left, Right, Bottom. For horizontal scrollers, you can align your scroll to start from the left or right. If you have a vertical scroller, you can use Top or Bottom alignments.
Start From: If your Movement Type is set to Endless, the Start From setting becomes available. By assigning an element from within your scroller to Start From, make it the first visible element in your scroller. For example, if you assign the 3rd element to Start From, your scroller will begin with the order 3, 4, 5, 1, 2 (assuming you have five elements in your scroller). Position for 3. element can be also specified by using the Selections of Positions variable.

```
AdvancedScrollerRef.Alignment = alignment.Left;
AdvancedScrollerRef.StartFromElement = element;
```

Padding



is a struct that consists of four integer variables: Top, Bottom, Left, and Right. Each of these variables can be assigned an integer value, allowing you to define padding for your scroller. These padding settings give you control over the spacing between your scroller's content and its boundaries.

```
AdvancedScrollerRef.Padding = new padding
{
    Top = 10,
    Bottom = 10,
    Left = 50,
    Right = 50
};
```

Spacing

is an integer variable which determines distance / space between elements.

```
AdvancedScrollerRef.Spacing = 20;
```

4. Others

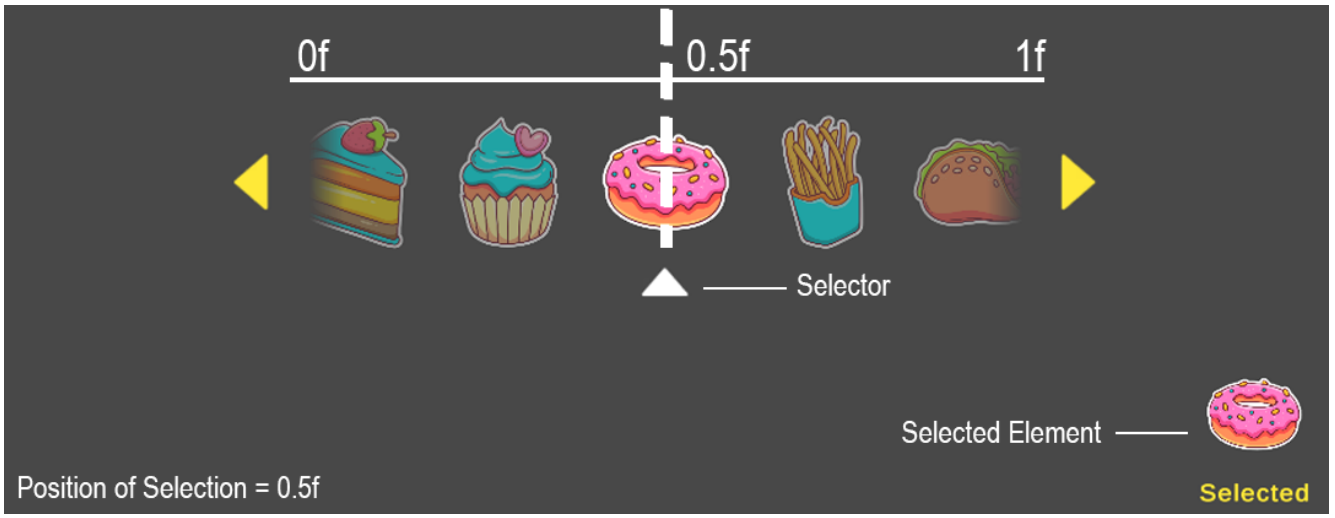
Position of Selection

A float value ranging from 0 to 1 that determines the position for content selection within the Advanced Scroller. For instance, if you set the "Position of Selection" to 0.5f, it will calculate the selection point based on the total width (or height for vertical scrollers) of the scroller's content. The selected element will be the one closest to the calculated position. Value also determines the position of the selector, which is a visual indicator highlighting the currently selected element within the scroller.

```
AdvancedScrollerRef.PositionOfSelection = 0.5f;
```

The "SelectedElement" method is a convenient way to retrieve the element within the scroller that corresponds to the specified position of selection.

```
AdvancedScrollerRef.SelectedElement();
```

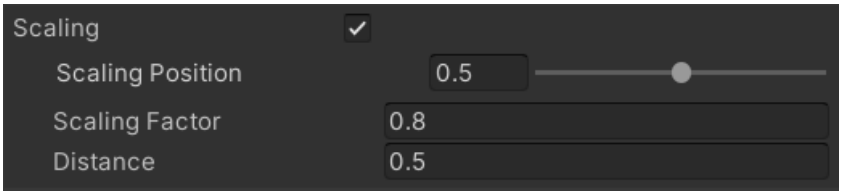


Inertia

Boolean value that determines whether the scroller should exhibit inertia-like behavior when scrolling. When "Inertia" is enabled (set to true), the scroller will continue to move for a short duration after the user releases the input (e.g., mouse or touch input) and then gradually come to a stop. If "Inertia" is set to true, you will also see an additional setting called "Deceleration Rate" in the inspector. This "Deceleration Rate" represents the speed at which the scroller gradually slows down during the inertia phase. A lower value will result in faster deceleration, bringing the scroller to a stop more quickly, while a higher value will cause a slower deceleration, extending the duration of the scroll.

```
AdvancedScrollerRef.Inertia = true;
AdvancedScrollerRef.DecelerationRate = 0.96f;
```

Scaling

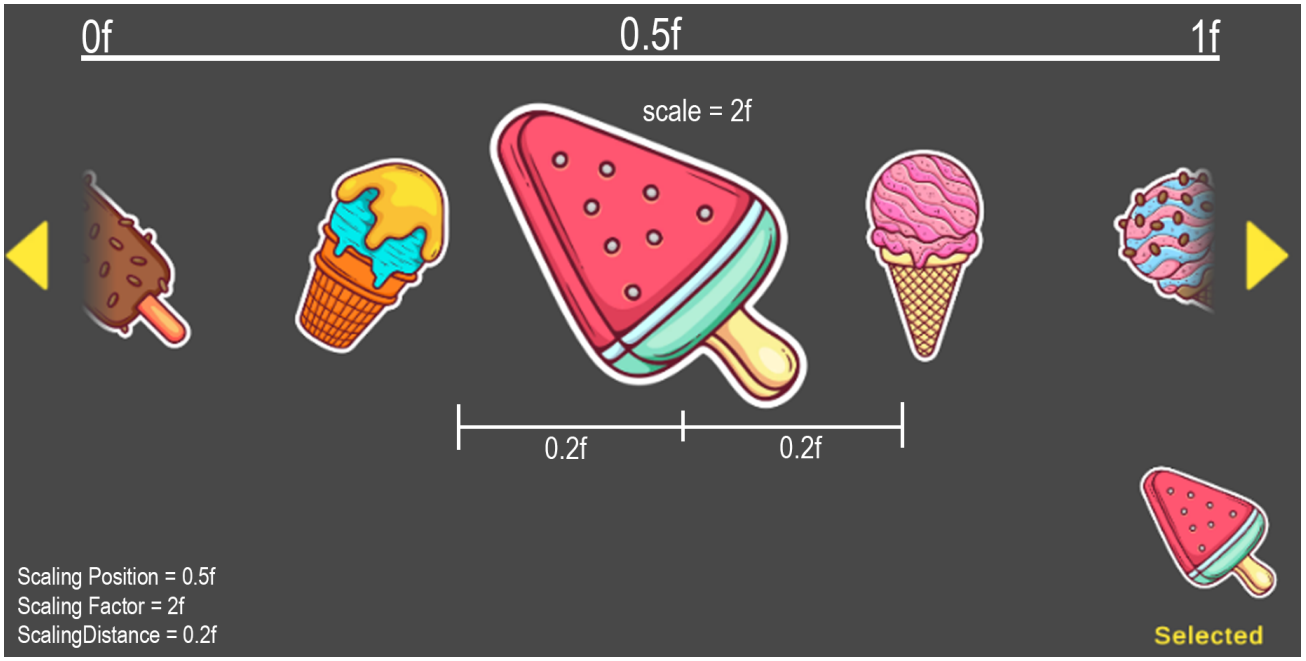


Boolean value that controls whether elements in the scroller should be scaled based on their positions relative to the "Scaling Position" setting. When "Scaling" is enabled (set to true), you can fine-tune various scaling parameters to create unique visual effects in your scroller.

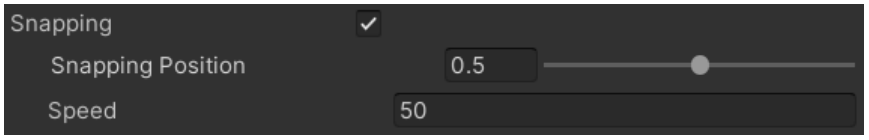
Here are the related settings for customizing the scaling behavior:

Scaling Position: This float value is clamped between 0 and 1 and determines the position along the Advanced Scroller where scaling is applied to elements. If you set "Scaling Position" to 0.5, scaling will be centered in the middle of the scroller. If you set it to 0 or 1, scaling will be applied at the start or end of the scroller, respectively.
Scaling Factor: This float value defines the scale applied to elements. For example, if you set scaling factors to 2, elements will appear twice as large. If you set the factor to 0.5, elements will appear half their original size.
Scaling Distance: Float value that determines how far from the "Scaling Position" the scaling effect is applied. Elements within this distance range from the "Scaling Position" will experience scaling, while elements outside this range will remain at their original size.

```
AdvancedScrollerRef.Scaling = true;
AdvancedScrollerRef.ScalingPosition = 0.5f;
AdvancedScrollerRef.ScalingFactor = 2f;
AdvancedScrollerRef.ScalingDistance = 0.2f;
```



Snapping

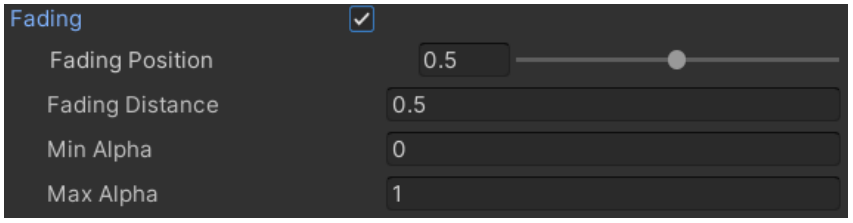


The "Snapping" setting controls whether elements snap to a specific position at the end of a drag. When snapping is enabled (set to true), you can configure the sub-settings for precise control over this snap behavior.

Snapping Position: This float value is clamped between 0 and 1 and determines where on the Advanced Scroller the elements will snap to. If you set the "Snapping Position" to 0.5, the snap point will be centered in the middle of the scroller. If you set it to 0 or 1, elements will snap to the start or end of the scroller, respectively.
Snapping Speed: This setting controls the speed at which elements snap to the specified position. It is float value, which influences the duration of the snap animation. Higher values result in faster snaps, while lower values produce slower, smoother snaps.

```
AdvancedScrollerRef.Snapping = true;
AdvancedScrollerRef.SnappingPosition = 0.5f;
AdvancedScrollerRef.SnappingSpeed = 20;
```

Fading

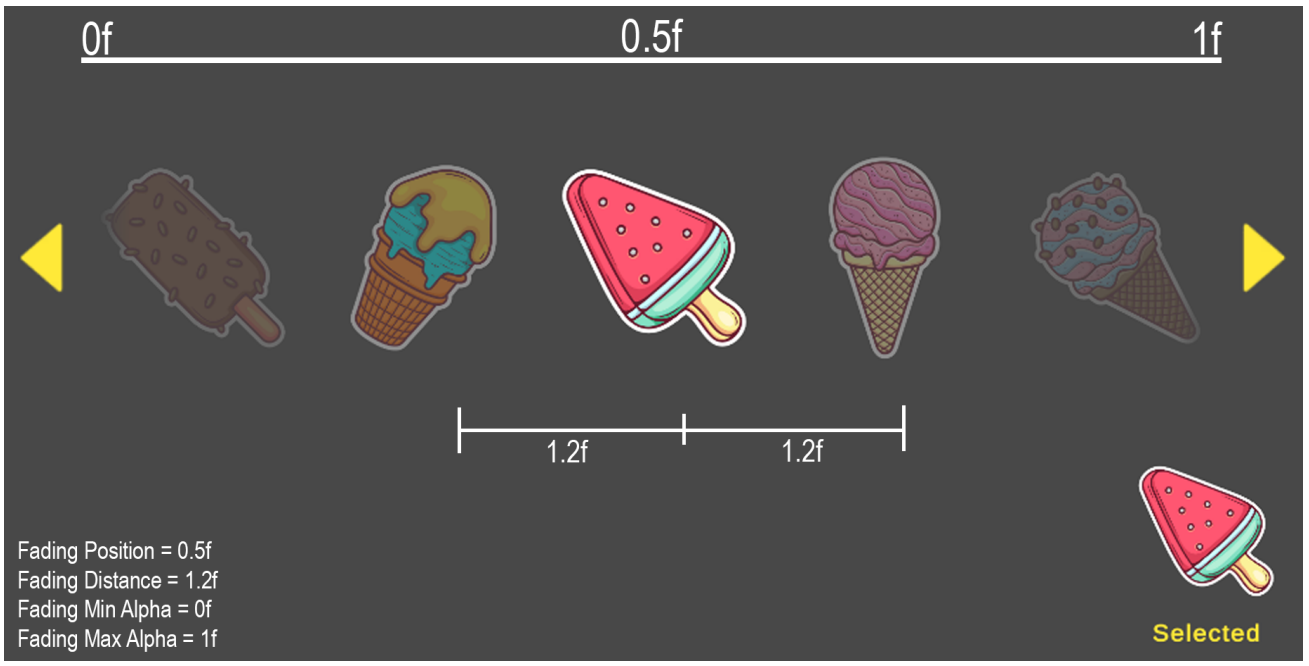


controls elements fade in/out as you scroll, offering visual feedback and enhancing user experience.

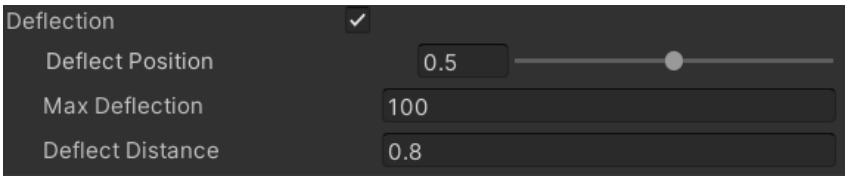
Fading Position: This float value is clamped between 0 and 1 and determines where on the Advanced Scroller the fading effect will be applied. If you set the "Fading Position" to 0.5, the fading effect will be centered in the middle of the scroller. If you set it to 0 or 1, fading will begin at the start or end of the scroller, respectively.
Minimum and Maximum Alpha: These settings allow you to control the opacity (alpha) of the elements based on their position within the scroller. You can specify a minimum and maximum alpha value to create a gradual fade-in effect as elements move from the "Fading Position" to their final position.
Fading Distance: This float value determines how far from the "Fading Position" the fading effect will be applied. The "Fading Distance" controls the range over which elements will transition from their minimum alpha to maximum alpha. A higher value will create a more extended fading effect, while a lower value will make the fading effect more localized.

```
AdvancedScrollerRef.Fading = true;
AdvancedScrollerRef.FadingPosition = 0.5f;
```

```
AdvancedScrollerRef.FadingDistance = 1.2f;
AdvancedScrollerRef.FadingMinAlpha = 0f;
AdvancedScrollerRef.FadingMaxAlpha = 1f;
```



Deflection



The "Deflection" setting in the Advanced Scroller allows you to control how elements respond to the scrolling interaction, offering a dynamic and engaging user experience. When "Deflection" is enabled (set to true), you can configure the sub-settings for precise control over the deflection effect.

Here are the sub-settings related to "Deflection":

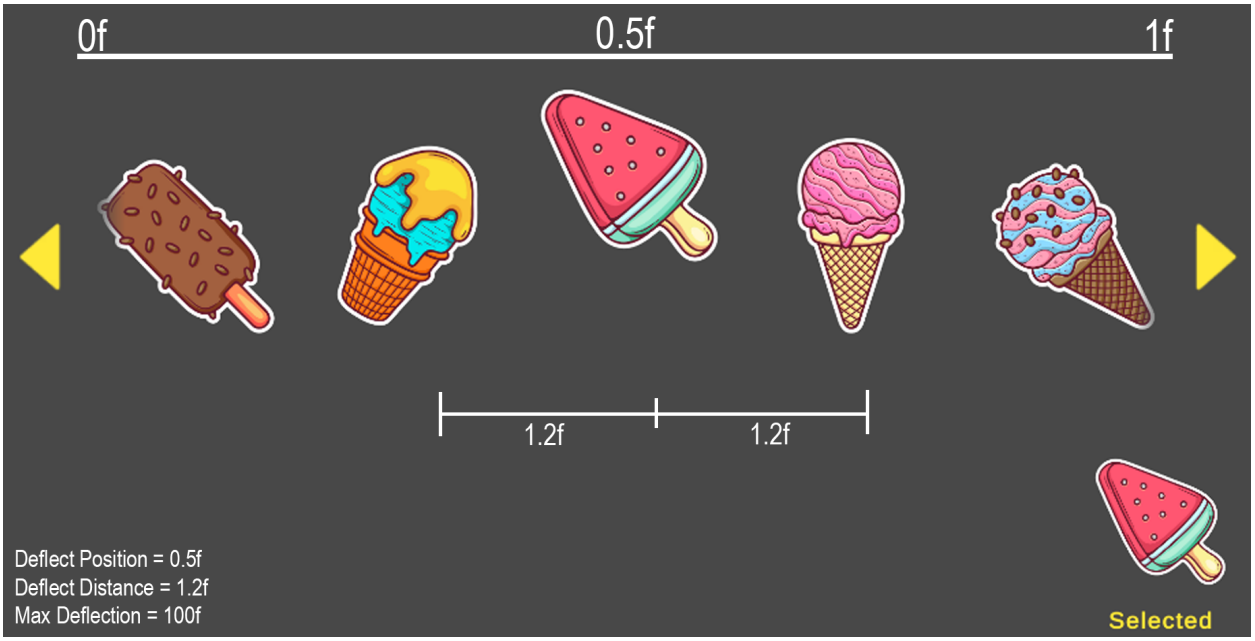
Deflection Position: This float value is clamped between 0 and 1 and determines where on the Advanced Scroller the deflection effect will be applied. If you set the "Deflection Position" to 0.5, the deflection effect will be centered in the middle of the scroller. If you set it to 0 or 1, deflection will begin at the start or end of the scroller, respectively.

Max Deflection (Y for Horizontal, X for Vertical): This setting determines the maximum deflection distance in the Y-axis for horizontal scrolling or in the X-axis for vertical scrolling. As users interact with the scroller, elements can deflect up or down (in horizontal mode) or left and right (in vertical mode) to create a dynamic response. The "Max Deflection" value controls the maximum amount of deflection allowed.

Deflection Distance: This float value determines how far from the "Deflection Position" the deflection effect will be applied. The "Deflection Distance" controls the

range over which elements will respond to the deflection effect. A higher value will create a more extended deflection range, while a lower value will make the deflection effect more localized.

```
AdvancedScrollerRef.Deflection = true;
AdvancedScrollerRef.DeflectPosition = 0.5f;
AdvancedScrollerRef.MaxDeflection = 100;
AdvancedScrollerRef.DeflectDistance = 0.8f;
```



Mask

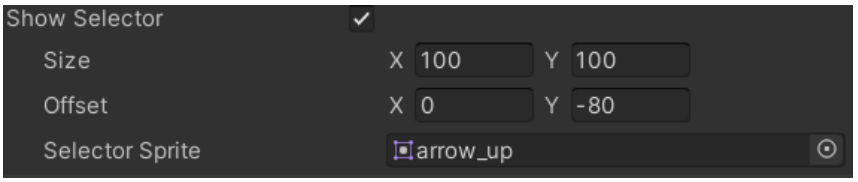
Allows you to create a visual mask that conceals any content outside of the scroller's boundaries, providing a sleek and contained look. When the "Mask" is enabled, all content outside the scroller will be hidden, and you can fine-tune the masking effect with additional padding and softness settings.

Padding (Left, Right, Top, Bottom): The padding settings determine the distance between the scroller's edge and the content inside the scroller. You can specify different padding values for the left, right, top, and bottom sides. Adjusting these settings can create space between the scroller's edge and the displayed content, enhancing the visual presentation.

Softness (X,Y): The softness settings control the smoothness of the mask's edges. Softness can add a gradual fade or blur effect to the mask, making it less visually abrupt.

```
AdvancedScrollerRef.Mask = true;
AdvancedScrollerRef.MaskSoftness = new Vector2Int(100, 100);
AdvancedScrollerRef.MaskPadding = new padding
{
    Left = 0,
    Right = 0,
    Top = 0,
    Bottom = 0
};
```

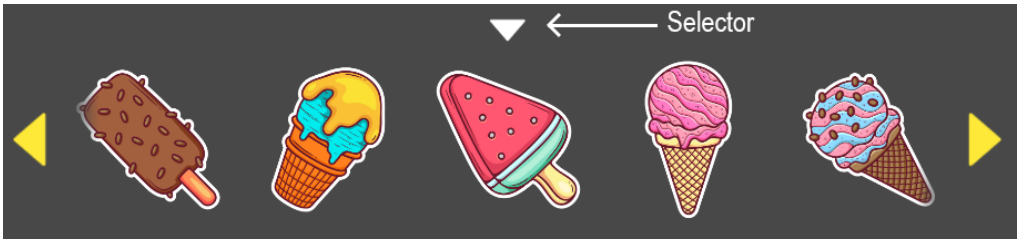
Show Selector



The "Show Selector" setting in the Advanced Scroller allows you to display a selector image that highlights the currently selected element within the scroller. Selector position is based on Position of Selection.

Size: You can specify the size of the selector image using a Vector2. This defines the dimensions (width and height) of the selector. Adjusting the size can help you customize the appearance of the selector, making it larger or smaller based on your design preferences.
Offset: The offset determines the position of the selector image relative to the selected element. By adjusting the offset with a Vector2, you can control the placement of the selector in relation to the selected element. This allows you to fine-tune the selector's positioning for the desired visual effect.
Selector Sprite: is a sprite that will be used as the selector. You can select the sprite you want to use from your project's assets. This sprite will be applied to the selector, making it visually distinct from the other elements.

```
AdvancedScrollerRef.ShowSelector = true;
AdvancedScrollerRef.SelectorOffset = Vector2.one;
AdvancedScrollerRef.SelectorSize = Vector2.one;
AdvancedScrollerRef.SelectorSprite = Sprite;
```



5. Useful methods

horizontalNormalizedPosition

This method gets or sets the horizontal scroll position of the Advanced Scroller as a normalized value between 0 (leftmost) and 1 (rightmost).

```
AdvancedScrollerRef.horizontalNormalizedPosition = 0.5f;
AdvancedScrollerRef.horizontalNormalizedPosition;
```

verticalNormalizedPosition

Similar to horizontalNormalizedPosition, this method gets or sets the vertical scroll position of the Advanced Scroller as a normalized value between 0 (top) and 1 (bottom). It provides programmatic control over the vertical scroll position.

```
AdvancedScrollerRef.verticalNormalizedPosition = 0.5f;
AdvancedScrollerRef.verticalNormalizedPosition;
```

ScrollToElement

This method is used to smoothly scroll the Advanced Scroller to a specific RectTransform element. The Element parameter should be the RectTransform of the target element. You can specify the duration for the scrolling animation, with a default value of 0 for an instant jump. Additionally, you can provide an OnComplete action to be executed when the scrolling is complete. This method is particularly useful for automatically scrolling to specific elements in response to user interactions or as part of your application's logic.

```
AdvancedScrollerRef.ScrollToElement(RectTransform Element, float duration = 0f,
Action OnComplete = null);
```

SelectedElement

This method is used to determine the element within the Advanced Scroller that is closest to a point defined by the PositionOfSelection variable. The position of selection is represented as a float value ranging from 0 to 1. The method calculates which element within the scroller is the nearest to this position and returns its RectTransform.

```
AdvancedScrollerRef.SelectedElement();
```

ScrollToNextElement

This method allows you to smoothly scroll to the next element within the Advanced Scroller. You can specify the duration of the scrolling animation using the duration parameter, and you can also provide an OnComplete action that will be executed once the scrolling is complete.

```
AdvancedScrollerRef.ScrollToNextElement(float duration = 0, Action OnComplete = null);
```

ScrollToPreviousElement

Similar to ScrollToNextElement, this method enables you to scroll to the previous element in the Advanced Scroller. You can set the duration of the scrolling animation using the duration parameter and define an OnComplete action to execute when the scroll concludes. These methods provide precise control over scrolling behavior and help you tailor the user experience in your Unity project.

```
AdvancedScrollerRef.ScrollToPreviousElement(float duration = 0, Action OnComplete = null);
```

DoScroll

The DoScroll method initiates scrolling in the Advanced Scroller for a defined duration. You can control the animation's speed and ease with the duration, speed, and Ease parameters. Specify the duration to set the total time of the scrolling animation. The speed parameter allows you to adjust the speed at which the scrolling occurs, providing control over the scroll velocity. The Ease parameter allows you to choose from a range of easing functions to influence the acceleration and deceleration of the scroll animation. Additionally, you can attach an Action to the callback parameter, which will be executed upon completion of the scrolling animation.

```
AdvancedScrollerRef.DoScroll(float duration = 2, float speed = 1, ease Ease = ease.Linear, Action callback = null);
```

LoadScroller

The LoadScroller method is used to trigger the loading animation in the Advanced Scroller. By invoking this method, you can initiate the loading animation according to the settings you've configured in the Inspector or by code (loading animation). These settings include the type of loading animation, whether it should play on start, a delay before the animation begins, the animation's duration, and the selected ease function. This method provides you with the ability to control and initiate loading animations programmatically in your Unity project.

```
AdvancedScrollerRef.LoadScroller(float delay);
```

Feedback and Support

I hope this documentation has been helpful in setting up and using the Advanced Scroller. If you encounter any issues, have questions, or need assistance, please don't hesitate to reach out. Your feedback is valuable in improving our product and providing the best possible experience.

Found a Bug or Need Help?

If you come across a bug, have questions, or require further assistance, please contact me via DiscordServer. I'll be happy to assist you in resolving any issues and making your experience with the Advanced Scroller as smooth as possible. Thank you for choosing the Advanced Scroller. I appreciate your support and I'm here to help you succeed in your projects.